LAPORAN PRAKTIKUM DEEP LEARNING

Attention dan Blok Transformer Sederhana

Cakrawala University

Kelompok 5

Identitas anggota kelompok

No.	Nama	NIM	Kontribusi
1	Tita Noviana	24120500011	25%
2	Titanio Yudista	24120500031	25%
3	Suci Fransisca Sisilia R	24120500008	25%
4	Fajar Dwiharjo	24130500010	25%

October 14, 2026

Contents

1 Pendahuluan **3**
- 1.1 Latar Belakang . . . 3
- 1.2 Tujuan Praktikum . . . 3
- 1.3 Batasan Laporan . . . 3

2 Konfigurasi dan Lingkungan **3**
- 2.1 Artefak Reproduksi . . . 3

3 Dasar Implementasi **4**
- 3.1 Scaled Dot-Product Attention . . . 4
- 3.2 Multi-Head Attention . . . 4
- 3.3 Masking . . . 4
- 3.4 Positional Encoding . . . 4

4 Verifikasi Perhitungan Manual **4**
- 4.1 Data Contoh . . . 4
- 4.2 Perhitungan Tangan . . . 5
- 4.3 Perbandingan dengan Program . . . 5

5 Multi-Head Attention dan Jumlah Parameter **5**
- 5.1 Alur dan Shape . . . 5

6 Masking **6**
- 6.1 Padding Mask . . . 6
- 6.2 Causal Mask . . . 6

7 Positional Encoding **7**

8 Transformer Block **7**
- 8.1 Struktur Block . . . 7

9 Uji Fungsional **8**
- 9.1 Data dan Model . . . 8
- 9.2 Konfigurasi Pelatihan . . . 8
- 9.3 Hasil Pelatihan dan Overfit Satu Batch . . . 8

10 Verifikasi Implementasi **9**

11 Pertanyaan Analisis **9**
- 11.1 Shape Tensor . . . 9
- 11.2 Scaling pada Attention . . . 10
- 11.3 Padding Mask dan Causal Mask . . . 10
- 11.4 Positional Encoding . . . 11
- 11.5 Layer Normalization . . . 11
- 11.6 Kesalahan Implementasi . . . 11
- 11.7 Kaitan dengan Model Recurrent . . . 11
- 11.8 Causal dan Bidirectional Attention . . . 11
- 11.9 Pengayaan NON-RPS . . . 11

12 Kesimpulan 11

A Perintah Menjalankan Program 12

B Kode atau Output Tambahan 12

1 Pendahuluan

1.1 Latar Belakang

Attention memungkinkan setiap posisi dalam sekuens menghitung bobot terhadap posisi lain berdasarkan kesesuaian query dan key, kemudian menggabungkan value secara berbobot. Transformer memanfaatkan mekanisme ini tanpa pemrosesan sekuens berulang, sehingga hubungan antarposisi dapat diperiksa melalui matriks attention.

1.2 Tujuan Praktikum

Praktikum ini bertujuan mengimplementasikan scaled dot-product attention, multi-head attention, masking, positional encoding sinusoidal, dan blok transformer sederhana. Implementasi diverifikasi melalui perhitungan manual, pemeriksaan shape, jumlah parameter, bobot attention, serta uji fungsional pada data sintetis.

1.3 Batasan Laporan

Laporan ini berfokus pada bagian Instruksi praktikum. Jawaban Pertanyaan Analisis disiapkan pada bagian khusus oleh anggota kelompok yang bertanggung jawab dan tidak diisi dalam template ini.

2 Konfigurasi dan Lingkungan

Table 1: Konfigurasi eksperimen

Parameter	Nilai
Framework	PyTorch
Seed	10
Format tensor	`(B,T,d_model)`
Batch size, B	2
Panjang sekuens, T	4
Dimensi model, d_{model}	8
Jumlah head, h	2
Dimensi per head, d_{head}	4
Dimensi feed-forward, d_{ff}	32
Aktivasi FFN	GELU
Optimizer	Adam
Learning rate	0.02

2.1 Artefak Reproduksi

Source code utama tersedia pada `attention_transformer.py`. Hasil verifikasi disimpan dalam `artifacts/verification_report.json`, ringkasan dalam `artifacts/verification_report.m` dan metrik pelatihan dalam `artifacts/training_metrics.csv`.

3 Dasar Implementasi

3.1 Scaled Dot-Product Attention

Scaled dot-product attention dihitung menggunakan:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{1}$$

Pada implementasi, skor dihitung dengan perkalian QK^T, dibagi dengan $\sqrt{d_k}$, lalu diproses oleh softmax pada dimensi key. Jika mask diberikan, posisi terlarang diisi dengan nilai sangat negatif sebelum softmax. Fungsi mengembalikan output, skor, dan bobot attention agar setiap tahap dapat diverifikasi.

3.2 Multi-Head Attention

Input diproyeksikan secara terpisah menjadi Q, K, dan V dengan dimensi akhir 8. Masing-masing tensor diubah dari $(B, T, 8)$ menjadi $(B, h, T, d_{\text{head}}) = (B, 2, T, 4)$. Attention dihitung secara paralel pada dua head, hasilnya digabung kembali menjadi $(B, T, 8)$, kemudian melewati output projection.

3.3 Masking

Padding mask diterapkan pada dimensi key untuk menghilangkan pengaruh token padding. Pada contoh batch, posisi padding adalah posisi keempat pada sampel pertama serta posisi ketiga dan keempat pada sampel kedua. Causal mask berupa matriks segitiga atas yang melarang setiap query melihat posisi masa depan. Mask diterapkan sebelum softmax sehingga bobot posisi terlarang menjadi nol. Implementasi juga menolak baris yang seluruh key-nya dilarang.

3.4 Positional Encoding

Positional encoding sinusoidal ditambahkan ke embedding input menggunakan

$$PE_{(p,2i)} = \sin\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right), \tag{2}$$

$$PE_{(p,2i+1)} = \cos\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right). \tag{3}$$

Implementasi menghasilkan encoding berukuran $(4, 8)$ dan menambahkannya secara broadcasting ke input berukuran $(2, 4, 8)$.

4 Verifikasi Perhitungan Manual

4.1 Data Contoh

Gunakan data wajib dari instruksi:

$$q = [1, 0, 1, 0], \qquad k_1 = [1, 0, 0, 0], \qquad k_2 = [0, 1, 0, 0],$$
$$v_1 = [1, 0], \qquad v_2 = [0, 1].$$

4.2 Perhitungan Tangan

$$
\begin{aligned}
qK^T &= [1, 0], \\
\frac{qK^T}{\sqrt{d_k}} &= [0.5, 0], \\
\text{softmax}([0.5, 0]) &= [0.622459,\ 0.377541], \\
\text{Attention}(q, K, V) &= [0.622459,\ 0.377541].
\end{aligned}
$$

4.3 Perbandingan dengan Program

Table 2: Perbandingan hasil manual dan program

Komponen	Hasil manual	Hasil program
Skor terskala	$[0.5, 0]$	$[0.5, 0]$
Bobot attention	$[0.622459, 0.377541]$	$[0.622459, 0.377541]$
Output	$[0.622459, 0.377541]$	$[0.622459, 0.377541]$
Error maksimum absolut	0.0	
Status	PASS	

5 Multi-Head Attention dan Jumlah Parameter

5.1 Alur dan Shape

Alur implementasi adalah proyeksi linear Q/K/V, split head, scaled dot-product attention, transpose dan concat antar-head, kemudian output projection. Untuk konfigurasi eksperimen, setiap proyeksi linear berukuran 8×8 dengan 8 bias.

Table 3: Jumlah parameter multi-head attention

Komponen	Bobot	Bias	Total
Proyeksi Q	$8 \times 8 = 64$	8	72
Proyeksi K	$8 \times 8 = 64$	8	72
Proyeksi V	$8 \times 8 = 64$	8	72
Output projection	$8 \times 8 = 64$	8	72
Total manual	256	32	**288**
Total program	**288**		**PASS**

6 Masking

6.1 Padding Mask

Mask padding yang digunakan adalah

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix},$$

dengan 1 menunjukkan key yang dilarang. Hasil program menunjukkan bobot terbesar pada seluruh posisi terlarang adalah 0.0. Jumlah setiap baris attention berada dalam toleransi 1.2×10^{-7} dari satu.

Table 4: Bukti padding mask

Pemeriksaan	Hasil
Posisi padding yang dilarang	Sampel 1: k_4; sampel 2: k_3, k_4
Bobot maksimum posisi terlarang	0.0
Deviasi maksimum jumlah baris dari 1	1.2×10^{-7}
Status	PASS

6.2 Causal Mask

Mask causal yang digunakan adalah

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Dengan demikian, query pada posisi t hanya dapat memperhatikan key pada posisi hingga t. Hasil program menunjukkan bobot maksimum pada posisi masa depan adalah 0.0 dan setiap baris valid tetap ternormalisasi.

Table 5: Bukti causal mask

Pemeriksaan	Hasil
Posisi masa depan yang dilarang	Segitiga atas
Bobot maksimum posisi terlarang	0.0
Deviasi maksimum jumlah baris dari 1	1.2×10^{-7}
Baris tanpa key valid ditolak	Ya
Status	PASS

7 Positional Encoding

Table 6: Verifikasi positional encoding

Pemeriksaan	Hasil
Shape encoding	$(4, 8)$
Shape setelah broadcasting	$(2, 4, 8)$
Posisi berbeda	Ya
Status	PASS

Nilai dua posisi pertama yang dihasilkan program adalah

$PE_0 = [0, 1, 0, 1, 0, 1, 0, 1]$,
$PE_1 = [0.841471, 0.540302, 0.099833, 0.995004, 0.0099998, 0.999950, 0.0010000, 1.000000]$.

Keduanya berbeda, sehingga pemeriksaan runtime untuk keberbedaan posisi dinyatakan lulus.

8 Transformer Block

8.1 Struktur Block

Block transformer terdiri atas multi-head attention, residual connection, layer normalization, feed-forward network, residual connection kedua, dan layer normalization kedua. Dalam kode, urutannya adalah

$$X \longrightarrow \mathrm{MHA}(X) \longrightarrow \mathrm{LayerNorm}(X + \mathrm{MHA}(X)) \longrightarrow \mathrm{FFN} \longrightarrow \mathrm{LayerNorm}(\cdot).$$

FFN menggunakan layer linear $8 \to 32$, aktivasi GELU, dan layer linear $32 \to 8$.

Table 7: Shape tensor pada transformer block

Tensor/tahap	Shape
Input embedding	$(2, 4, 8)$
Q, K, V sebelum split head	$(2, 4, 8)$
Q, K, V sesudah split head	$(2, 2, 4, 4)$
Matriks skor per head	$(2, 2, 4, 4)$
Bobot attention per head	$(2, 2, 4, 4)$
Keluaran head sebelum concat	$(2, 2, 4, 4)$
Setelah concat	$(2, 4, 8)$
Keluaran MHA	$(2, 4, 8)$
Keluaran FFN	$(2, 4, 8)$
Keluaran akhir block	$(2, 4, 8)$

Table 8: Jumlah parameter transformer block

Komponen	Jumlah parameter
Multi-head attention	288
Linear FFN pertama	288
Linear FFN kedua	264
Dua layer normalization	32
Total manual	**872**
Total program	**872**

9 Uji Fungsional

9.1 Data dan Model

Uji fungsional menggunakan klasifikasi sekuens pendek menjadi dua kelas pada data sintetis. Label dibentuk secara deterministik dari tanda rata-rata fitur pertama sepanjang sekuens. Data train terdiri atas 32 sekuens dan data test terdiri atas 64 sekuens. Setiap sekuens memiliki panjang 4 dan dimensi fitur 8. Model menambahkan positional encoding, memproses input dengan satu transformer block, melakukan mean pooling pada dimensi sekuens, lalu menghasilkan dua logit kelas.

9.2 Konfigurasi Pelatihan

Model dilatih selama 100 epoch menggunakan Adam dengan learning rate 0.02 dan cross-entropy loss. Seed acak ditetapkan ke 10. Data train diproses sebagai satu batch penuh pada setiap epoch sehingga penurunan loss dan kenaikan akurasi dapat dipakai sebagai bukti kemampuan overfit pada batch kecil.

9.3 Hasil Pelatihan dan Overfit Satu Batch

Table 9: Ringkasan hasil uji fungsional

Metrik	Nilai
Loss awal	0.734517
Loss akhir	0.000168
Akurasi train akhir	1.0000
Akurasi test	0.8750
Overfit satu batch	PASS

Metrik per epoch tersedia pada `artifacts/training_metrics.csv`. Kurva loss dan akurasi dapat dibuat dari file tersebut jika visualisasi diperlukan.

10 Verifikasi Implementasi

Table 10: Ringkasan pemeriksaan implementasi

Pemeriksaan	Status	Bukti
Perhitungan manual attention	PASS	Error maksimum 0.0
Jumlah parameter MHA	PASS	288 = 288
Jumlah parameter block	PASS	872 = 872
Shape tensor	PASS	Assertion runtime
Padding mask	PASS	Bobot terlarang 0.0
Causal mask	PASS	Bobot masa depan 0.0
Positional encoding	PASS	Shape dan posisi berbeda
Overfit satu batch	PASS	Akurasi train 1.0000

Seluruh pemeriksaan dilakukan dengan assertion runtime pada `attention_transformer.py`. Ringkasan numerik lengkap dapat dilihat pada `artifacts/verification_report.json`; ringkasannya tersedia pada `artifacts/verification_report.md`.

11 Pertanyaan Analisis

Bagian ini sengaja disediakan sebagai template dan dikerjakan oleh anggota kelompok lain. Jangan mengisi bagian ini dari laporan implementasi utama sebelum jawaban anggota terkait digabungkan.

11.1 Shape Tensor

Untuk konfigurasi $d_model = 8$, $h = 2$ (sehingga $d_head = d_model/h = 4$), $T = 4$, dan $B = 2$, shape tensor pada setiap tahap `MultiHeadAttention` adalah sebagai berikut. Nilai ini sesuai dengan hasil pemeriksaan program pada `shape_and_parameter_verification` di `attention_transformer.py` dan Tabel 10.

Table 11: Shape tensor pada setiap tahap multi-head attention

Tensor	Shape	Keterangan
Input	$(B, T, d_model) = (2, 4, 8)$	masukan blok transformer
Query/Key/Value (sebelum split head)	$(2, 4, 8)$	keluaran proyeksi linear q, k
Query/Key/Value (setelah split head)	$(B, h, T, d_head) = (2, 2, 4, 4)$	setelah `view` + `transpose(`
Matriks skor $QK^\top/\sqrt{d_k}$	$(B, h, T, T) = (2, 2, 4, 4)$	skor sebelum softmax, per h
Bobot attention (setelah softmax)	$(2, 2, 4, 4)$	setiap baris berjumlah 1
Output per head (weights $\times V$)	$(2, 2, 4, 4)$	sebelum concat head
Output setelah concat head	$(B, T, d_model) = (2, 4, 8)$	`transpose` + `view` kembali
Keluaran akhir MHA	$(2, 4, 8)$	setelah `output_projection`

Dimensi batch B dan panjang sekuens T dipertahankan di seluruh tahap, sedangkan dimensi fitur berpindah dari $d_model = 8$ menjadi $h \times d_head = 2 \times 4$ saat kepala attention dipisah, lalu kembali menjadi $d_model = 8$ setelah hasil setiap kepala digabungkan (concat) dan melewati `output_projection`. Matriks skor dan bobot attention berbentuk (B, h, T, T) karena setiap query dari tiap kepala dibandingkan dengan seluruh T key pada kepala yang sama.

11.2 Scaling pada Attention

Skor attention $QK^\top$ dibagi dengan $\sqrt{d_k}$ (pada implementasi ini $\sqrt{d_head} = \sqrt{4} = 2$) karena, jika elemen q dan k diasumsikan memiliki rata-rata 0 dan varians 1 serta saling bebas, hasil dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ memiliki varians yang tumbuh linear terhadap d_k. Tanpa pembagian ini, magnitudo skor akan semakin besar seiring bertambahnya d_k, sehingga skor antar key bisa memiliki selisih yang sangat besar. Pembagian dengan $\sqrt{d_k}$ menormalkan varians skor agar tetap mendekati 1 berapa pun nilai d_k, sehingga skor berada pada rentang yang stabil sebelum masuk ke softmax.

Jika pembagian tersebut dihilangkan, skor mentah $QK^\top$ cenderung memiliki magnitudo besar (terutama untuk d_k yang besar). Softmax dari skor bermagnitudo besar akan *saturasi*: distribusi bobot menjadi sangat tajam dan mendekati one-hot pada key dengan skor tertinggi, sementara bobot pada key lain mendekati nol. Pada daerah saturasi ini, turunan fungsi softmax mendekati nol, sehingga gradien yang mengalir balik ke Q dan K saat training menjadi sangat kecil (vanishing gradient). Akibatnya proses optimisasi melambat atau bahkan berhenti belajar, dan perilaku ini akan semakin parah ketika d_model atau d_head diperbesar.

11.3 Padding Mask dan Causal Mask

Kedua jenis mask pada `attention_transformer.py` bekerja dengan mekanisme yang sama, yaitu memberi nilai sangat negatif (`torch.finfo(scores.dtype).min`) pada skor di posisi terlarang sebelum softmax dijalankan (fungsi `scaled_dot_product_attention`), sehingga bobot attention pada posisi tersebut menjadi 0 setelah softmax. Perbedaan keduanya terletak pada *posisi mana yang dilarang* dan *alasan pelarangannya*:

- **Padding mask** melarang attention membaca token padding (token isian yang ditambahkan agar panjang sekuens dalam satu batch sama) pada dimensi *key*. Mask ini sama untuk seluruh posisi query dalam satu sekuens — pada implementasi ini direpresentasikan sebagai `padding_mask[:, None, None, :]` berbentuk $(B, 1, 1, T)$ yang di-broadcast ke seluruh head dan seluruh baris query. Tujuannya adalah mencegah token padding yang tidak memiliki makna ikut memengaruhi hasil representasi token lain.

- **Causal mask** melarang query pada posisi t membaca key pada posisi $t' > t$ (masa depan), terlepas dari apakah token tersebut padding atau bukan. Mask ini dibentuk dengan matriks segitiga atas `torch.triu(..., diagonal=1)` berbentuk (T, T) yang sama untuk seluruh batch dan head. Tujuannya adalah menjaga sifat autoregresif, yaitu prediksi pada posisi t hanya boleh bergantung pada token $1, \ldots, t$.

Bukti bahwa kedua mask bekerja dengan benar ditunjukkan oleh `masking_verification`: bobot attention maksimum pada posisi terlarang untuk padding mask dan causal mask

sama-sama 0.0 (lihat Tabel 10), sementara jumlah bobot pada setiap baris yang valid tetap 1.0.

Jika causal mask tidak diterapkan pada model autoregresif, query pada posisi t dapat melihat key pada posisi $t' > t$, yaitu token yang seharusnya belum "diketahui" model pada saat prediksi dibuat. Saat training, model dapat "mencontek" dengan memanfaatkan informasi token masa depan untuk memprediksi token saat ini, sehingga loss training terlihat rendah namun model tidak benar-benar mempelajari distribusi bersyarat $P(x_t \mid x_{<t})$ yang sesungguhnya. Pada saat inferensi, token masa depan belum tersedia (dihasilkan satu per satu), sehingga terjadi ketidaksesuaian antara kondisi training dan inferensi (train–inference mismatch) yang membuat kualitas prediksi menurun drastis dibandingkan hasil pada data training.

11.4 Positional Encoding

[TODO: Jawaban Pertanyaan Analisis 4.]

11.5 Layer Normalization

[TODO: Jawaban Pertanyaan Analisis 5.]

11.6 Kesalahan Implementasi

[TODO: Jawaban Pertanyaan Analisis 6.]

11.7 Kaitan dengan Model Recurrent

[TODO: Jawaban Pertanyaan Analisis 7.]

11.8 Causal dan Bidirectional Attention

[TODO: Jawaban Pertanyaan Analisis 8.]

11.9 Pengayaan NON-RPS

[TODO: Jawaban Pertanyaan Analisis 9.]

12 Kesimpulan

Bagian Instruksi berhasil diimplementasikan dan diverifikasi. Perhitungan manual scaled dot-product attention menghasilkan error maksimum absolut 0.0. Multi-head attention memiliki 288 parameter dan transformer block memiliki 872 parameter, sesuai dengan perhitungan manual. Padding mask dan causal mask menghasilkan bobot nol pada seluruh posisi terlarang. Positional encoding menghasilkan shape yang sesuai, yaitu $(4, 8)$ sebelum broadcasting dan $(2, 4, 8)$ setelah ditambahkan ke input. Pada uji fungsional, loss turun dari 0.734517 menjadi 0.000168 dan akurasi train mencapai 1.0000, sehingga kriteria overfit satu batch terpenuhi. Akurasi test yang diperoleh adalah 0.8750 pada data sintetis yang digunakan.

A Perintah Menjalankan Program

```
python attention_transformer.py
```

B Kode atau Output Tambahan

Source code lengkap tersedia pada `attention_transformer.py`. Output verifikasi yang digunakan untuk mengisi laporan ini tersedia pada `artifacts/verification_report.json`, sedangkan metrik per epoch tersedia pada `artifacts/training_metrics.csv`.